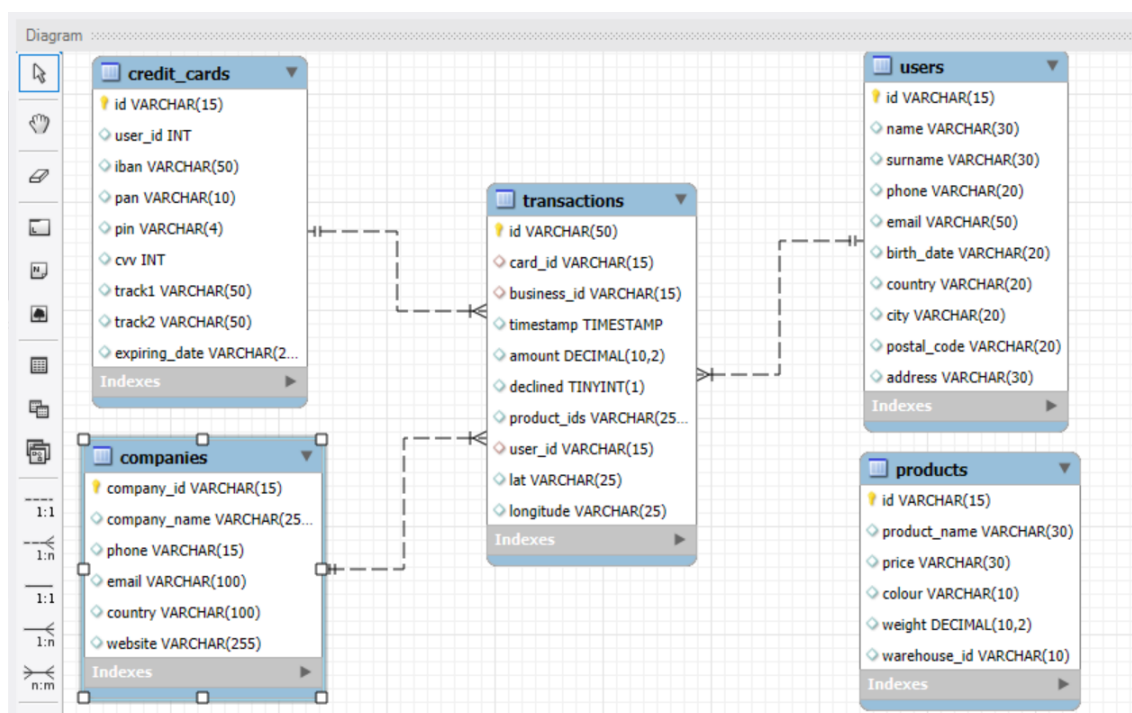
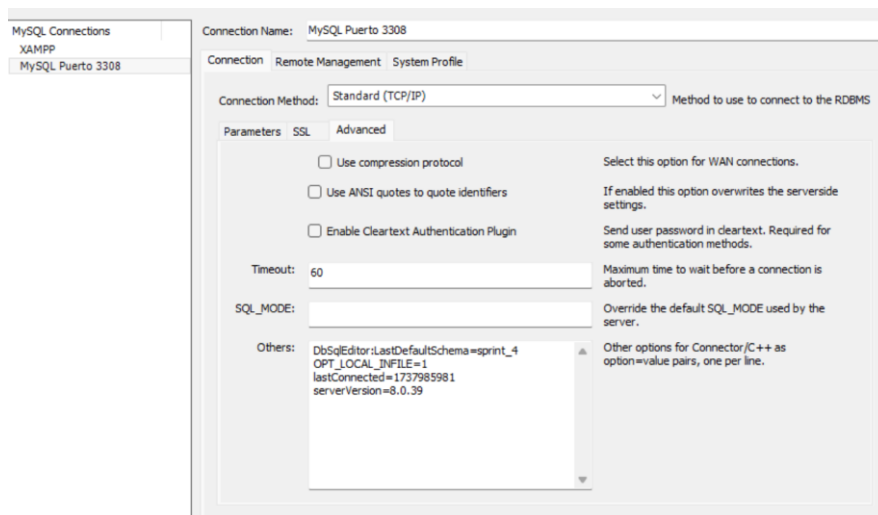


NIVEL 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes

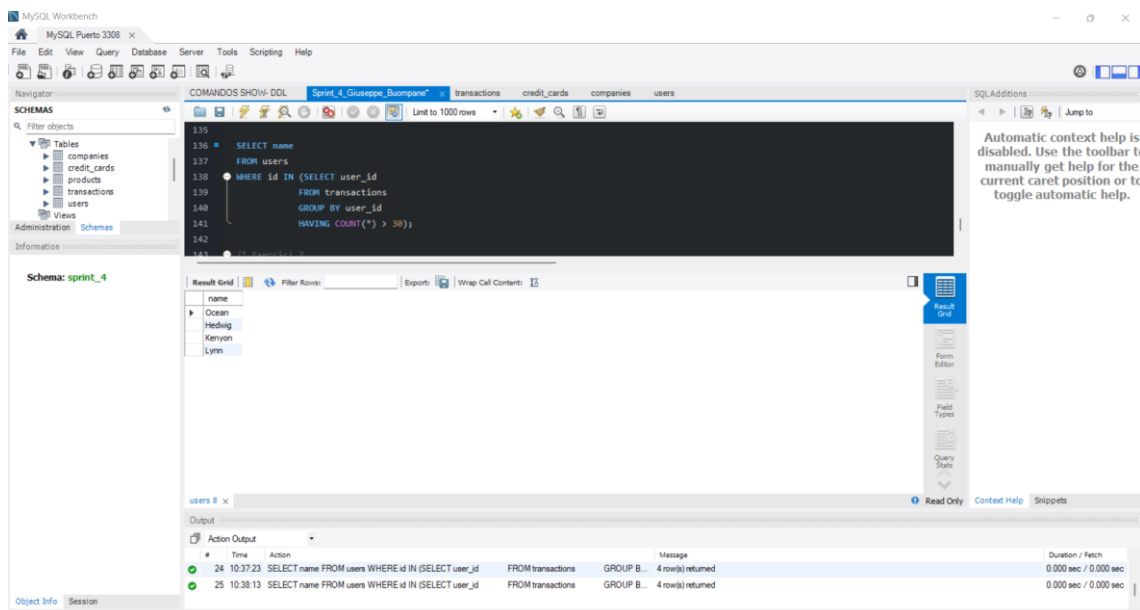


Aquí creamos la BBDD Sprint_4 y luego creamos tablas y acto seguido cargamos datos, por ultimo creamos transactions, cargamos datos y creamos las FK/constraints . Creamos 1 sola tabla users, cargando los datos de los 3 países ya que tienen los mismos campos/datos. Dependiendo de los separados de los CSV y el entorno en los que fueron creados, tenemos que ajustar el código de la carga. Configurar el workbench añadiendo en manage server connections, en el box others: OPT_LOCAL_INFILE =1 + ejecutar el comando SET GLOBAL local_infile=1 para permitir la carga local de los archivos en caso de que esté deshabilitada. La tabla products está creada pero no está relacionada aún con la BBDD.



- Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 30 transaccions utilitzant almenys 2 taules.



- Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists the database structure. The 'COMANDOS SHOW-DLL' pane displays the following SQL query:

```
153 select credit.iban, comp.company_name, round(avg(trans.amount),3) as promedio_trans
154 from transactions as trans
155 join credit_cards as credit
156 on trans.card_id = credit.id
157 join companies as comp
158 on trans.business_id = comp.company_id
159 where comp.company_name = "Donec Ltd"
160 group by 1,2;
```

The 'Result Grid' shows the following data:

iban	company_name	promedio_trans
PT87806228135092429456346	Donec Ltd	203.715

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
123	20:06:56	SELECT credit.iban, comp.company_name, ROUND(AVG(trans.amou...	1 row(s) returned	0.000 sec / 0.000 sec
124	20:08:07	select credit.iban, comp.company_name,round(avg(trans.amount),3) as promedio...	1 row(s) returned	0.000 sec / 0.000 sec

NIVEL 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les últimes tres transaccions van ser declinades i genera la següent consulta:

Exercici 1

Quantes targetes estan actives?

The screenshot shows the MySQL Workbench interface. The 'Schemas' pane on the left lists the database structure. The 'COMANDOS SHOW-DLL' pane displays the following SQL query:

```
192 GROUP BY card_id
193 )
194 SELECT COUNT(*) AS Num_tarjetas_activas
195 FROM estado_tarjetas
196 WHERE estado_tarjeta = "activa"
197
198
199
```

The 'Result Grid' shows the following data:

Num_tarjetas_activas
275

The 'Output' pane shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
83	19:22:30	WITH transacciones_ordenadas AS (SELECT card_id AS card_id,trans.decline...	1 row(s) returned	0.016 sec / 0.000 sec
84	19:25:41	SELECT * FROM sprint_4.credit_cards LIMIT 0, 1000	275 row(s) returned	0.000 sec / 0.000 sec

con el WITH creamos 2 tablas temporales sobre las que podemos trabajar, con la window function : row_number() crea una nueva columna y a cada fila le asigna un numero a partir de 1.

Partition by: divide los datos (las transacciones) por cada id, con el order by timestamp desc, obtenemos las transacciones ordenadas por la más reciente, de este modo sabemos que por cada grupo id, el numero 1,2,3 son las ultimas 3 transacciones. En la tabla estado_tarjeta, utilizamos un CASE para crear una nueva columna con condición para establecer el estado de cada tarjeta(id).

las tarjetas que tienen la suma = 3 (1+1+1) del campo declined y 3 del count de filas son las tarjetas inactivas, las demás, recogidas por el else , están inactivas.

NIVEL 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids.

Genera la següent consulta:

The screenshot shows the MySQL Workbench interface. The 'COMANDOS SHOW-DDL' tab displays the SQL commands for creating the 'prod_transactions' table and inserting data from the 'transactions' table. The 'Result Grid' shows the first row of data inserted into 'prod_transactions'.

```
287 CREATE TABLE IF NOT EXISTS prod_transactions (
288   id INT AUTO_INCREMENT PRIMARY KEY,
289   transaction_id VARCHAR(50),
290   product_id VARCHAR(15)
291 );
292 INSERT INTO prod_transactions (transaction_id, product_id)
293 SELECT
294   trans_id,
295   TRIM(SUBSTRING_INDEX(SUBSTRING_INDEX(trans.product_ids, ',', n.n), ',', -1)) AS product_id
```

id	transaction_id	product_id	cantidad
2048	B4793303-EDF5-4F53-39CB-AA6A2877CDF8	37	1

The 'Output' tab shows the execution of the SQL commands, with messages indicating the number of rows returned for each step.

Hemos generado una tabla intermedia llamada prod_transactions para poder conectar la tabla products con la BBDD existente. La tabla intermedia contiene un campo ID por cada registro que hemos generado con un autoincrement (identificador único por cada registro), un campo transaction_id (FK), y un campo product_id (FK) que refleja cada uno de los ids que hemos extraído del campo producto_ids de la tabla transacciones. Al crear la tabla, hemos estado barajando la idea de crear un campo 'cantidad', al final lo hemos añadido tras la creación de la tabla, como tarea adicional, especificando mis

consideraciones y realizando una nueva consulta, la tarea en cuestión se encuentra detallada en el file sql.

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
259 FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE CASCADE;
260
261 /*Exercici 1
262 Necessitem conèixer el nombre de vegades que s'ha venut cada producte*/
263
264 SELECT prod.id, prod.product_name, count(prodtra.product_id) prod_vendidos
265 FROM prod_transactions as prodtra
266 Join products as prod
267 on prodtra.product_id = prod.id
```

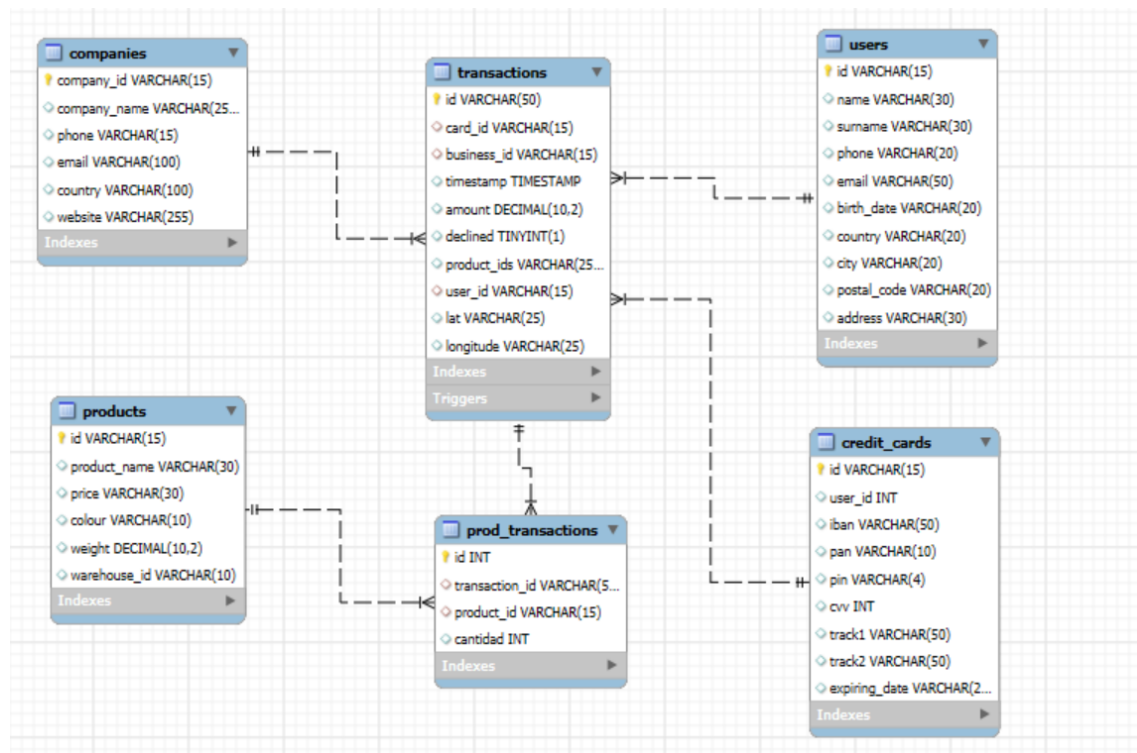
The Results tab shows the following data:

id	product_name	prod_vendidos
23	riverlands north	68
67	Winterfell	68
79	Direwolf riverlands the	66
2	Tarly Stark	65
43	duel	65

The Output tab shows the execution log:

#	Time	Action	Message	Duration / Fetch
109	18.23.52	SELECT prod.id, prod.product_name, count(prodtra.product_id) prod_vendidos ...	26 row(s) returned	0.000 sec / 0.000 sec
110	18.23.58	SELECT prod.id, prod.product_name, count(prodtra.product_id) prod_vendidos ...	26 row(s) returned	0.016 sec / 0.000 sec

Diagrama final (modelo dimensional copo de nieve) y tareas adicionales



The screenshot displays a database management interface with the following components:

- Navigator:** Shows a tree view of database objects including Columns, Indexes, Foreign Keys, Triggers, and users.
- COMANDOS SHOW DDL:** A text editor showing the SQL code for creating a trigger named `actualiza_prod_transaccion_AI`. The code includes comments in Spanish explaining the trigger's logic.
- Result Grid:** A table showing the results of the trigger execution. It has columns for `id`, `transaction_id`, `product_id`, and `cantidad`. The data shows a successful insert into `prod_transactions`.
- Output:** A log of actions performed, including the execution of the trigger and the resulting messages.

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Hemos creado un trigger para que también se actualice automáticamente `prod_transactions` cada vez que se inserte un nuevo registro en Transactions. Insertamos un registro y verificamos ejecución (detalles con código en el file sql).